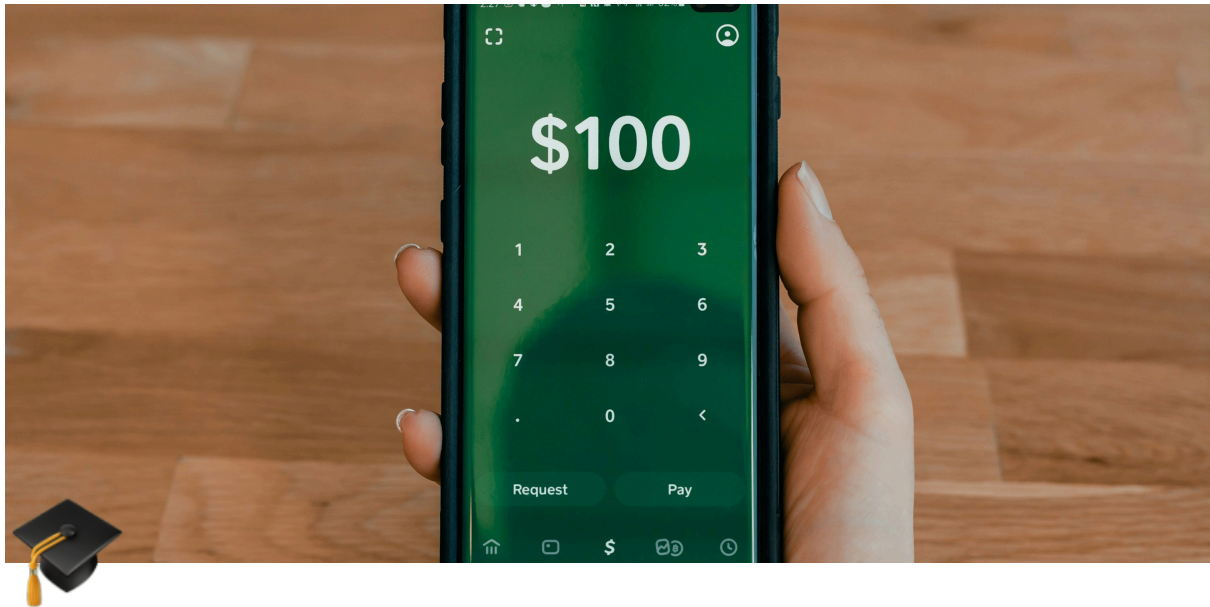




Full Stack Engineer Challenge

Kindo School Payments Feature Challenge

Introduction

Thank you for your interest in joining Kindo as a Senior Full Stack Developer! This coding challenge is designed to give you an opportunity to showcase your skills with our tech stack while solving a problem similar to what you might encounter in this role.

The challenge focuses on building a simple school payment feature that interfaces with a legacy payment API. You'll create both a frontend interface and backend services to support payment collection for a school activity.

This is a chance for you to demonstrate both your engineering and ai skills. We recognise and want people to use the latest coding agents and assistants, but also know if those tools are hallucinating or going awry.

We expect the time investment on this challenge to be around 1-2 hours.

Please reach out to us if you have any questions.

Problem Statement

A school needs to collect payments for an upcoming field trip. Parents need to be able to:

- View trip details (date, cost, location)
- Register their child for the trip
- Make a payment through an online form
- Receive confirmation of payment

You'll need to build a small full-stack application that handles this flow while integrating with our legacy payment processing API.

Technical Requirements

Backend (Python/Django/FastAPI)

- Create a Django or Fast API that:
 - Provides endpoints to fetch trip details
 - Accepts and validates payment submissions
 - Interfaces with the legacy payment API
 - Stores transaction records
 - Returns appropriate responses to the frontend
- Legacy API Integration:
 - You must integrate with our legacy payment processing API (details below)
 - The API expects a specific format and returns success/failure responses that you'll need to handle

Frontend (React)

- Create a small React application that:
 - Displays the field trip details
 - Includes a registration form for child's information
 - Provides a payment form with validation

- Shows appropriate success/error messages
- Has a clean, responsive design (Tailwind CSS encouraged but not required)

Legacy Payment API Interface

```
# This simulates the legacy payment API that you'll need to integrate with#
You should include this in your project and call it as needed
import time
import random
from dataclasses import dataclass
from typing import Optional
@dataclass
class PaymentResponse:
    success: bool
    transaction_id: Optional[str] = None
    error_message: Optional[str] = None
class LegacyPaymentProcessor:
    """
    Legacy payment processor that accepts payments and provides responses.

    This is a simulation of an external API you'd need to work with.
    """

    def process_payment(self, payment_data):
        """
        Process a payment with the following required fields:
        - student_name: str
        - parent_name: str
        - amount: float
        - card_number: str (must be 16 digits)
        - expiry_date: str (format: MM/YY)
        - cvv: str (must be 3 digits)
        - school_id: str
        - activity_id: str
        """
        required_fields = [
            'student_name', 'parent_name', 'amount',
```

```

        'card_number', 'expiry_date', 'cvv',
        'school_id', 'activity_id'
    ]

    # Validate all required fields exist
    for field in required_fields:
        if field not in payment_data:
            return PaymentResponse(
                success=False,
                error_message=f"Missing required field: {field}"
            )

    # Validate card number (must be 16 digits)
    card_num = payment_data['card_number'].replace(' ', '')
    if not (card_num.isdigit() and len(card_num) == 16):
        return PaymentResponse(
            success=False,
            error_message="Invalid card number. Must be 16 digits."
        )

    # Validate expiry date format (MM/YY)
    if not self._validate_expiry_format(payment_data['expiry_date']):
        return PaymentResponse(
            success=False,
            error_message="Invalid expiry date format. Use MM/YY."
        )

    # Validate CVV (must be 3 digits)
    if not (payment_data['cvv'].isdigit() and len(payment_data['cvv']) ==
3):
        return PaymentResponse(
            success=False,
            error_message="Invalid CVV. Must be 3 digits."
        )

    # Validate amount is positive
    if not isinstance(payment_data['amount'], (int, float)) or payment_data
['amount'] <= 0:

```

```

        return PaymentResponse(
            success=False,
            error_message="Payment amount must be a positive number."
        )

# Simulate processing time
time.sleep(1.5)

# Simulate occasional payment failures
if random.random() < 0.1:# 10% chance of failure
    return PaymentResponse(
        success=False,
        error_message="Payment declined by processor. Please try again."
    )

# Generate a transaction ID for successful payments
transaction_id = f"TX-{int(time.time())}-{random.randint(1000, 9999)}"

    return PaymentResponse(
        success=True,
        transaction_id=transaction_id
    )

def _validate_expiry_format(self, expiry):
    """Validate the expiry date is in MM/YY format."""
    if not isinstance(expiry, str) or len(expiry) != 5:
        return False

    if expiry[2] != '/':
        return False

    month, year = expiry.split('/')

    if not (month.isdigit() and year.isdigit()):
        return False

    month_num = int(month)

```

```
if month_num < 1 or month_num > 12:  
    return False  
  
return True
```

Constraints/Requirements

- **Tech Stack:**
 - Backend: Python with Django
 - Frontend: React
 - Relational Database: Whatever free relational db you prefer, e.g. SQLite is fine for this challenge
- **Must-Have Features:**
 - Functional payment flow from frontend to backend
 - Integration with the legacy payment API
 - Form validation on both frontend and backend
 - Error handling and user feedback
 - At least basic styling for a good user experience
- **Nice-to-Have Features:**
 - Responsive design with Tailwind CSS
 - Unit tests for critical components
 - Error logging
 - Loading states during payment processing
 - Receipt generation
- **Evaluation Criteria:**
 - Code quality and organization (including regular commits)
 - Problem-solving approach
 - Error handling
 - User experience considerations
 - Documentation

Bonus Points

- **Deployment:** Deploy your solution to a free hosting service (Vercel, Netlify, Heroku, AWS Free Tier, etc.)
- **AI Integration:** Document how you leveraged AI tools (like GitHub Copilot, ChatGPT, cursor) to help with your development process
- **React :** Include a simple responsive web view to support mobile devices.

Submission Guidelines

- Create a GitHub/BitBucket repository for your solution
- Include a README with:
 - Setup instructions
 - Overview of your solution
 - Any assumptions or design decisions
 - Description of how you used AI tools (if applicable)
- Be prepared to walk through your code and explain your decisions during the interview

.

We're excited to see your approach to this challenge! Remember, we're not just looking for a working solution but also evaluating your coding style, problem-solving skills, and how you approach integrating with legacy systems.

Good luck!